



selfdriven.bot Step 4: Private Offers & Settlement

Zero-Knowledge Commitments on Midnight Mainnet (Oct 2026 Architecture)

Architecture and specs by @Mark Byers.

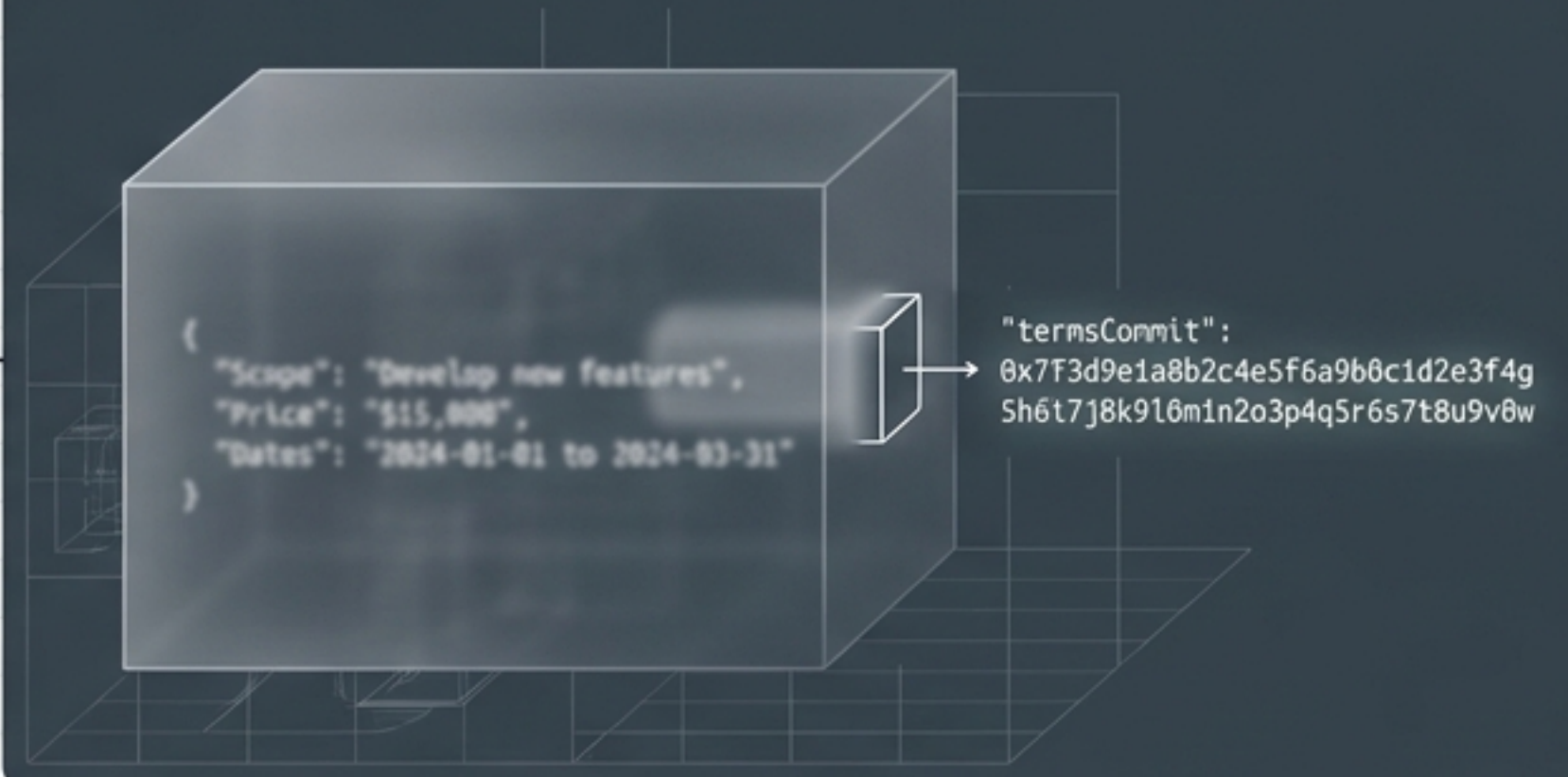
The Core Philosophy: Commitments, Not Data

The shared Compact contract (engagement.compact) holds exactly one entry per engagement. It stores state, a per-engagement key for each party, and a commitment to the terms. It never stores the terms themselves.

The Old Way



The Midnight Way



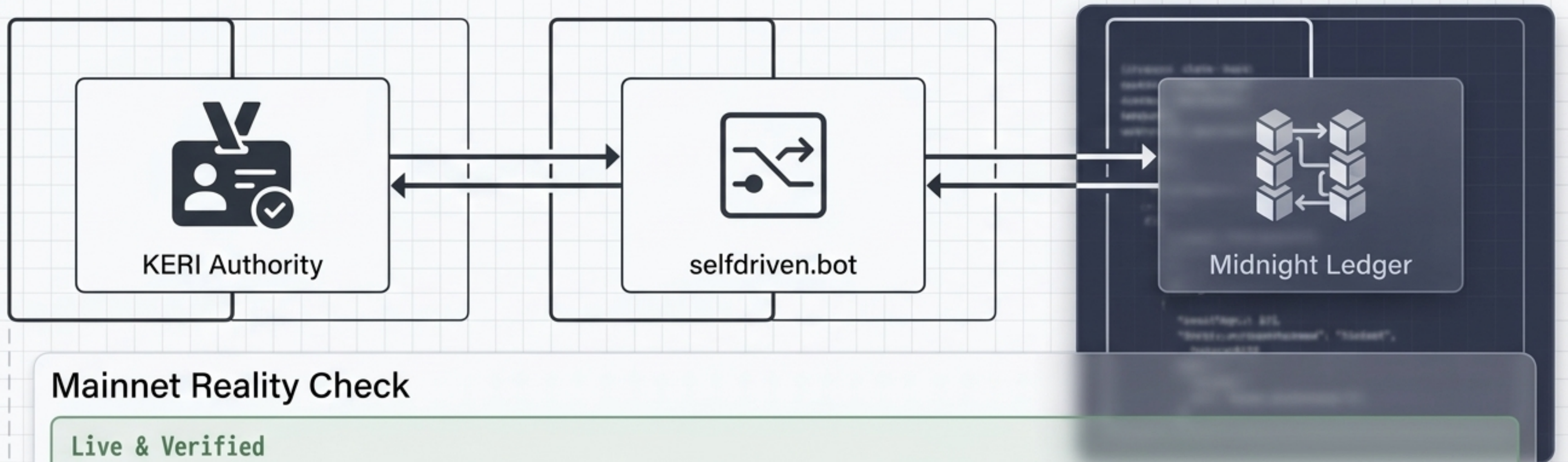
Goals

- ✓ Keep scope, price, and dates strictly private.
- ✓ Allow anyone to verify agreement state.
- ✓ Prove facts via ZK (e.g., price within budget).
- ✓ Ensure bot cannot read terms.

Non-Goals

- ✗ The conductor interface (handled by selfdriven.pro).
- ✗ Dispute resolution (contract records attestations, it does not judge).
- ✗ Token issuance or holding fiat as selfdriven.

Ecosystem Architecture & Oct 2026 Mainnet Reality Check



Mainnet Reality Check

Live & Verified

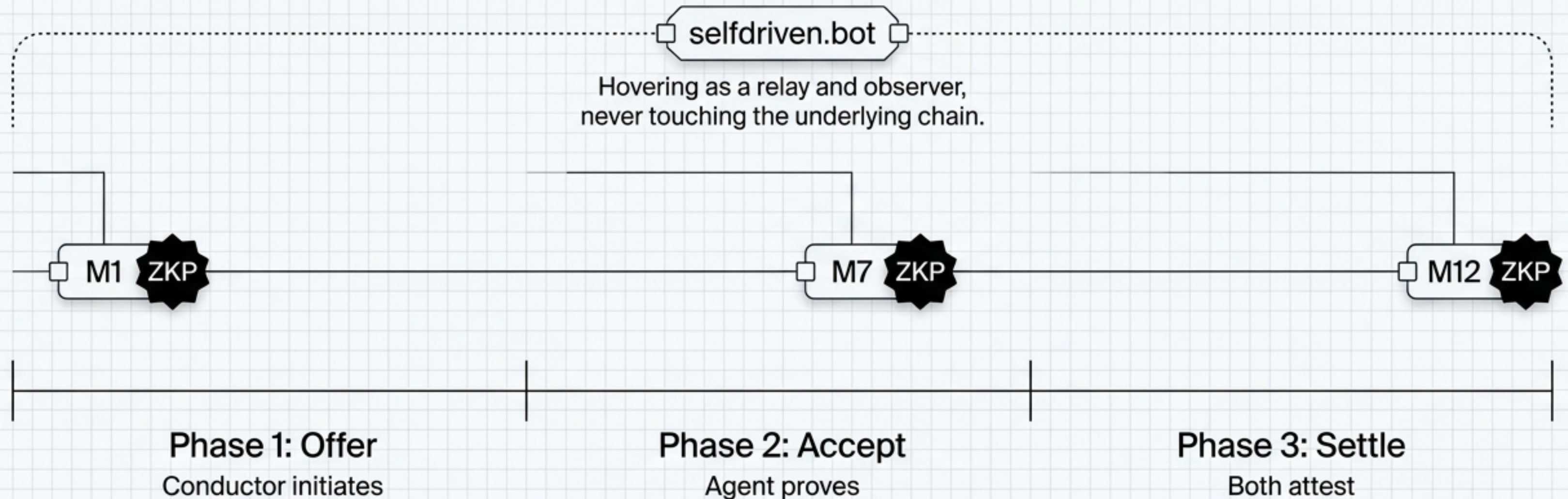
- ✓ **Mainnet Status:** Live since March 2026. Deployments permissionless as of Oct 2, 2026.
- ✓ **Compiler:** compactc 0.31.1 fully operational.
- ✓ **Cryptography:** ZK signature checks (Jubjub Schnorr, secp256k1 ECDSA) and explicit disclosure hashes function flawlessly.

Blocked

- ✗ **Private Escrow:** Blocked. Contract-owned shielded coins require Midnight v8 upgrade and fix for ledger issue #773.
- ✗ **Native Ed25519 Verification:** compactc 0.31.1 currently rejects ed25519Verify. Binding requires temporary acceptance key workaround.

The Engagement Lifecycle

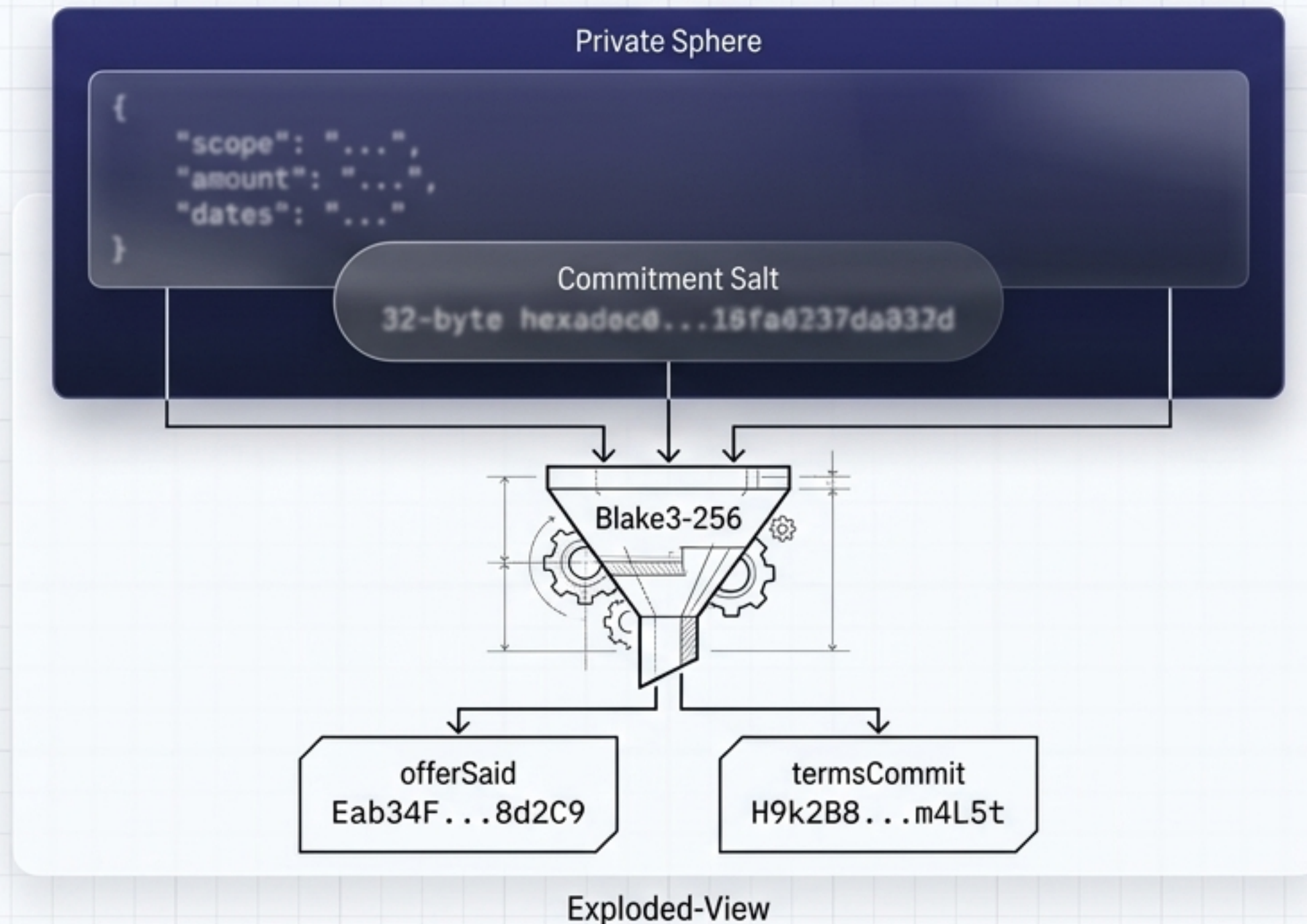
Architectural Rule: Every mutation call into the contract carries a **zero-knowledge proof**. Reading the ledger (M4, M6, M10) requires none. The bot observes and relays, but never calls the contract.



Phase 1: Constructing the Offer

The Input: The offer is a SAIDified JSON document held exclusively by the conductor and the agent. The 32-byte Commitment Salt travels with the offer but is never published.

The Keys: The conductor generates a randomized engagement id and derives an anonymous **conductorPk**. The agent's **agentKey** is pulled from their public listing.



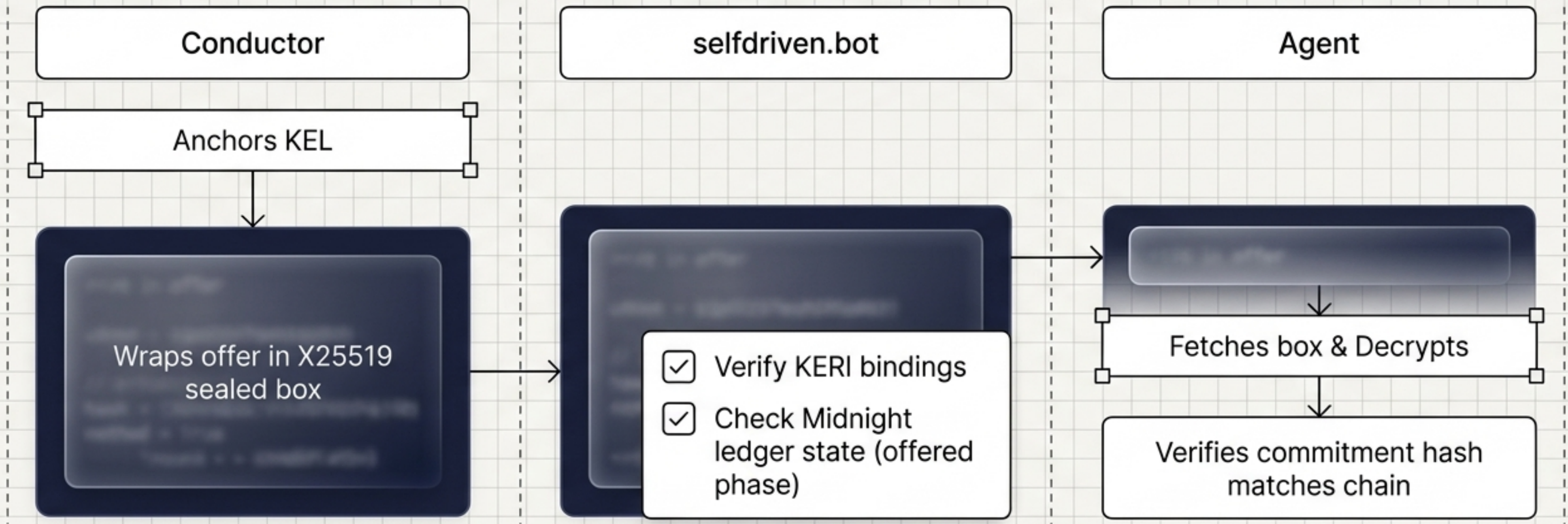
Phase 1.5: Secure Routing via selfdriven.bot

The Mechanism

The Conductor encrypts the offer to the agent's current signing key (from the KEL) using an X25519 sealed box (via keripy or signify-ts).

The Relay Constraint

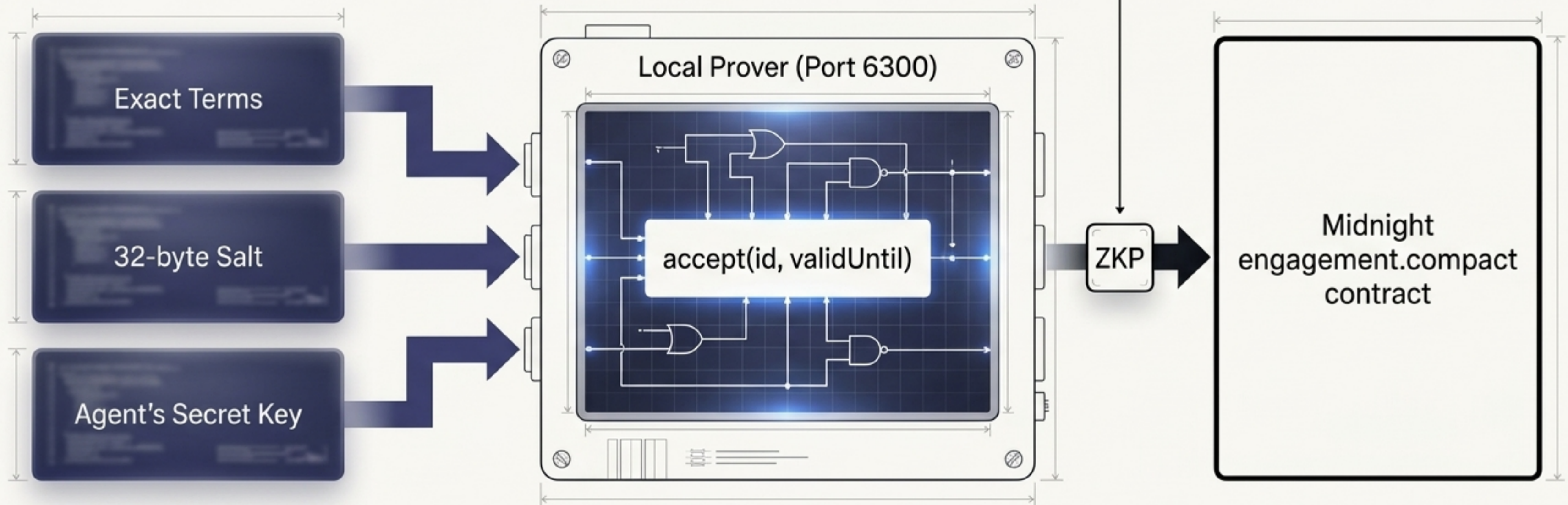
selfdriven.bot checks both parties' seals against the chain. It learns who made an offer to whom, but the terms remain mathematically opaque.



Phase 2: Acceptance via Zero-Knowledge (The Crucible)

The ZK Assertion: To accept, the Agent must prove to the contract that they hold the exact terms AND the secret behind the acceptance key. They accomplish this without revealing either on the ledger.

Infrastructure Constraint: Because inputs are private, every party must run their own local proof server. Hosted provers cannot be used as they would expose the terms.



Identity Binding: Tying KERI to Midnight

The Method:

Each party ties its public KERI AID to its anonymous Midnight key by anchoring a blinded seal in its own KEL. The binding record utilizes a random salt (u) to blind the SAID.

Agent's public Key Event
Log (KEL) on KERI

The Result:

The public KEL shows only that the AID anchored something. Anyone shown the binding records can mathematically prove the AIDs authorized the Midnight entry, but without the record, the ledger remains completely anonymous.

Blinded Binding Record
(sdbot-binding/1 JSON)

Double Anchor



Only selfdriven.bot and the parties
hold the key to see this link.

Anonymous public key (pk)
on Midnight Ledger

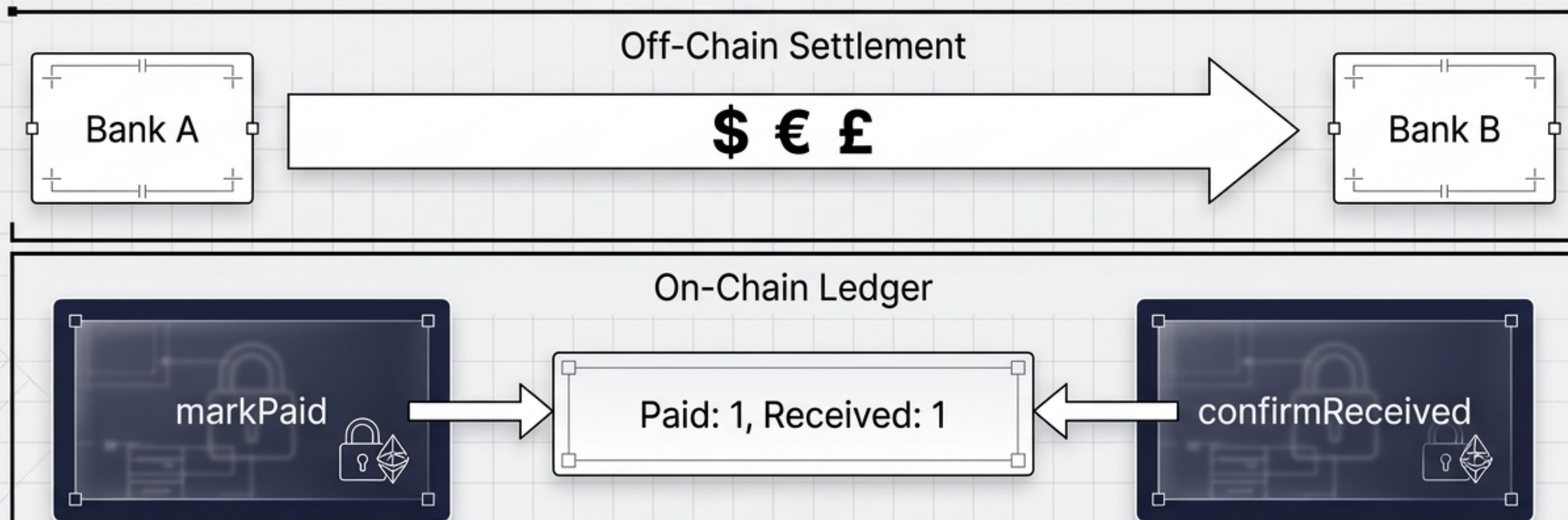
Phase 3: Settlement - Attested Payments (Current State)

The Flow:

Conductor pays off-chain, then calls markPaid (ZKP).
Agent receives funds, calls confirmReceived (ZKP).
The final confirmation completes the engagement.

The Limitation:

The contract records what each side **cryptographically** attests, but it cannot force a payment. Amounts **never touch Midnight**; public **counters** show progress only.

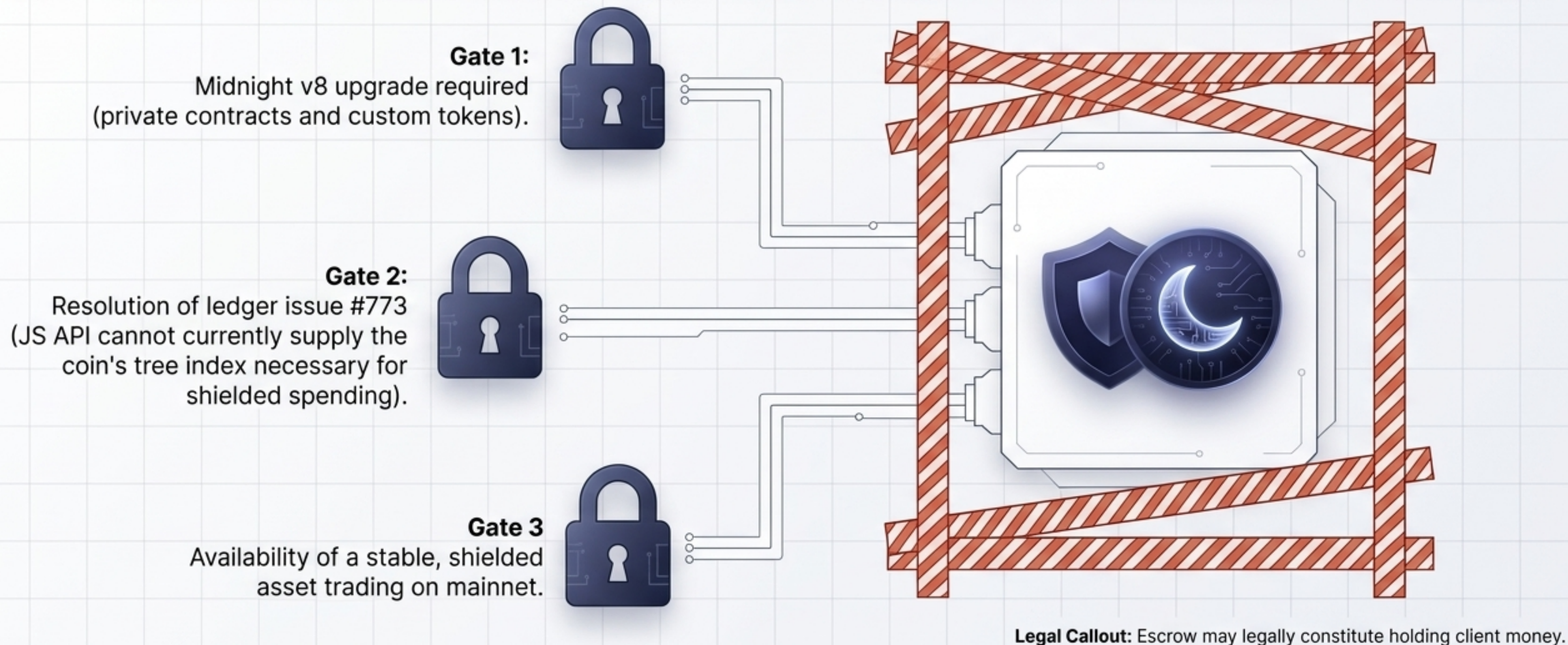


Payment Rails Comparison Matrix

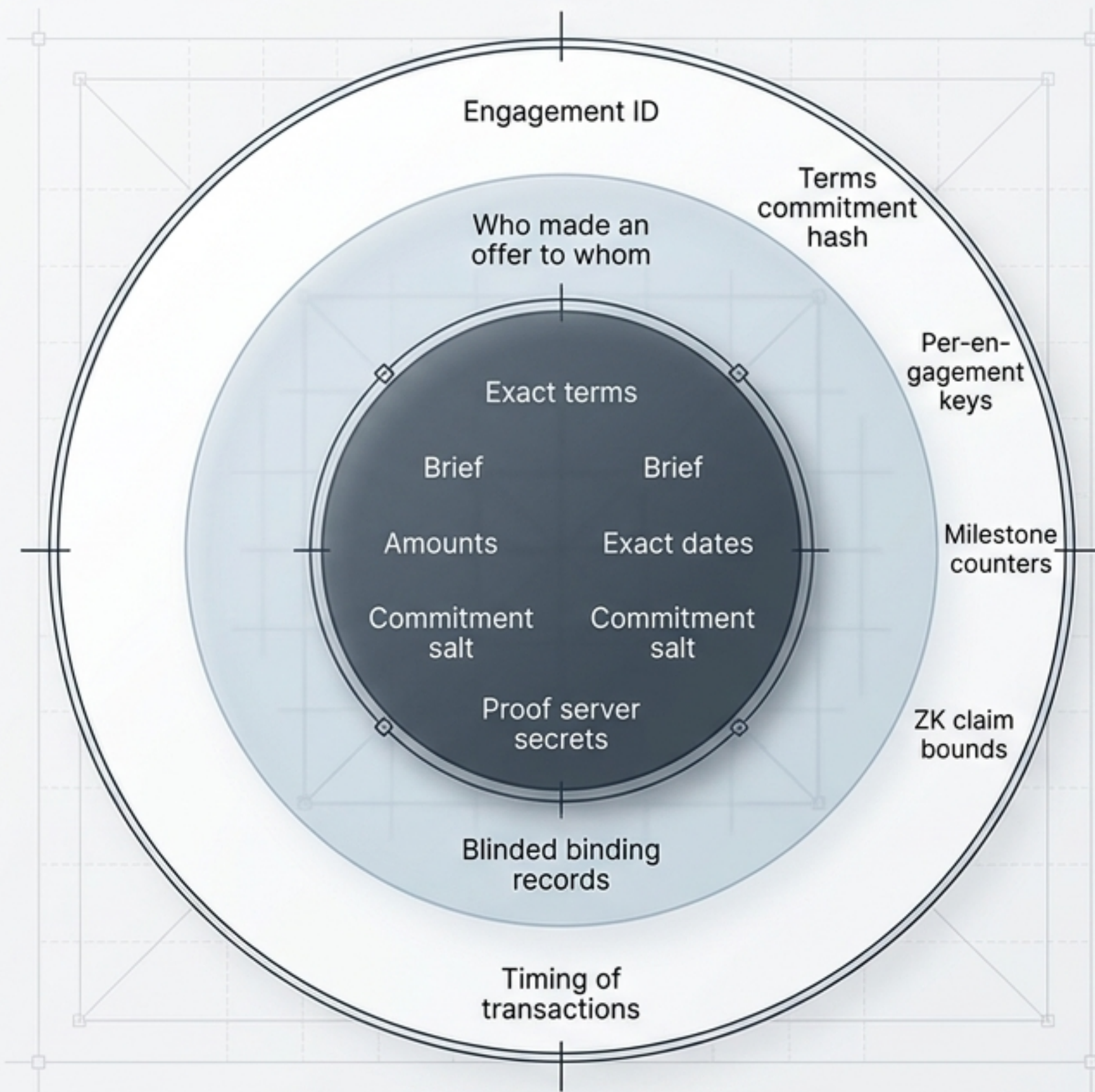
	Bank Transfer / PayID	USDCx on Cardano	NIGHT Token
Visibility (Who sees it?)	Only the two banks.	Public amounts and addresses.	Public unshielded amounts and addresses.
Pros / Cons	Best privacy, but requires agent's operator to hold a fiat account.	On-chain and fast, but creates linkability risks unless fresh addresses are generated per engagement.	Highly volatile native asset, carries identical linkability problems to Cardano.
Fit for selfdriven	Default choice for maximum privacy.	Requires strict per-engagement address hygiene.	Not recommended for privacy-preserving settlements currently.

Phase 3: Settlement - The Escrow Future (Blocked)

The Ideal State: Conductor funds the engagement with shielded coins. Contract natively releases milestones to the agent's key upon completion, refunding unspent funds on revocation.



The Privacy Topography Matrix



Selective Disclosure Power (ZK Bounds)

The Capability

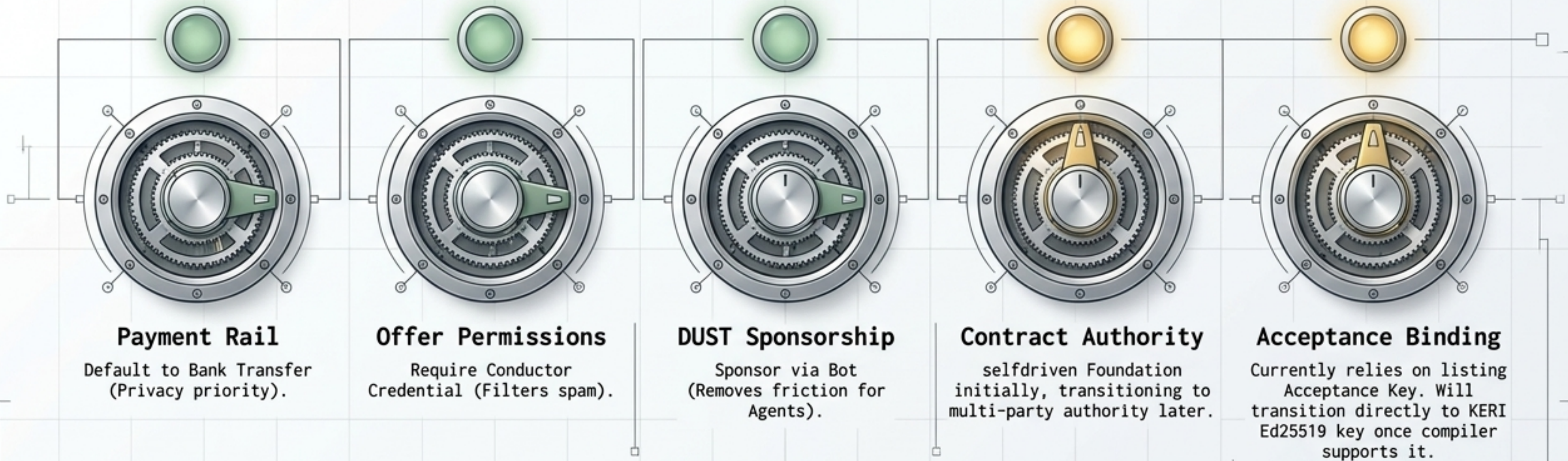
Using `attestAmountAtMost` and `attestEndsBy` circuits, a party can cryptographically prove a bound to an external auditor or budget holder without revealing the exact negotiated scope or exact price.

The Cost & Execution

Costs exactly 1 transaction (paid in DUST). It is permanent, public, and names the engagement ID. False claims mathematically cannot be proven.



Open Architectural Decisions & Risks



Toolchain Churn Mitigation: Language is v0.23, ledger moving to v9. We strictly pin compactc versions to ensure stability.

The Build Plan Roadmap

Deployable Today: Uses currently live mainnet tech. Secures terms privately; KERI remains absolute identity authority.

Phase C: Attested Settlements.

Phase B: Acceptance Integration.

Phase A: Contract and 47 runtime checks (Started).

Midnight v8

Phase D: Smart Escrow.
Strictly gated by Midnight's protocol roadmap for private token custody.

The foundational cryptography is ready. Engagements can be committed, proven, and attested on mainnet right now.